Incorporating Attention Residuals into Diffusion Transformer Policies for Robot Learning

1. Introduction

Diffusion-based visuomotor policies have become a key paradigm in robot learning, achieving outstanding performance in manipulation tasks by modelling action generation as a denoising process [Chi et al., RSS 2023]. Recently, the **Diffusion Transformer Policy** further replaced the CNN denoising backbone with a Transformer, significantly enhancing the model's expressive power and scalability.

However, this model still employs traditional additive residual connections, making it difficult to adaptively utilise features from different hierarchical levels. The latest work, **Attention Residuals (AttnRes)**, points out that fixed residuals lead to the gradual dilution of information in deep networks, and proposes achieving dynamic information selection across depth dimensions by applying attention-weighted averaging to outputs from historical layers.

This project introduces AttnRes into the denoising network of the Diffusion Transformer Policy and evaluates its performance on the ManiSkill simulation task. **We hypothesise that different diffusion stages correspond to different hierarchical information requirements, and that AttnRes can achieve adaptive feature selection through a deep attention mechanism, thereby enhancing policy performance.**

2. Problem Formulation

Let the robot's policy be, given an observation $\mathbf{o}$, the action sequence $\mathbf{a} = a_{t=1}^{T}$ the conditional distribution. The Diffusion Policy learns this distribution through an iterative denoising process:

$$\mathbf{a}_{k-1} = f_\theta(\mathbf{a}_k, k, \mathbf{o})$$

Here, k denotes the diffusion time step, and f_θ denotes a denoising network with a Transformer as its parameter. The Transformer consists of L stacked blocks, each of which performs:$$\tilde{\mathbf{x}}l = \mathbf{x}l - 1 + \mathrm{Attn}(\mathrm{LN}(\mathbf{x}_{l-1}))$$

$$\mathbf{x}_l = \tilde{\mathbf{x}}_l + \mathrm{FFN}(\mathrm{LN}(\tilde{\mathbf{x}}_{l-1}))$$

Key issue: Additive residuals force the output of each layer to directly accumulate all previous activations with equal weights. In a standard Pre-LN setup, this leads to diminishing returns at deeper layers. This issue is particularly pronounced in diffusion denoising—changes at diffusion time step k can result in markedly different noise semantics, and an adaptive information routing mechanism may offer greater advantages.

3. The proposed method

We propose replacing the standard residual connections in the Diffusion Transformer Policy denoising backbone with **Attention Residuals (AttnRes)**. With minimal structural changes and without significantly increasing model complexity, we implement an adaptive information routing mechanism across depth dimensions. To ensure training stability, we initialise the query vectors to values close to zero, causing the model's initial behaviour to approximate that of standard residual connections, thereby achieving a smooth transition.

Integration into the Diffusion Transformer Policy. Table 1 Comparison of DiT Block structural modifications:

Component	Original structure	modified
Self-attention residual in the DiT block	`x = x + Attn(LN(x))`	Compute a weighted sum of $h_0, \ldots, h_l^*$ using AttnRes
The FFN residual in the DiT block	`x = x + FFN(LN(x))`	As above, weighted sum of AttnRes
Cross-attention (conditional injection)	Unchanged	Unchanged
Visual/language encoder	Unchanged	Unchanged

In practice, all that is required is to maintain a `hidden_states_history` list, and add a small linear projection for each layer to generate $\mathbf{q}_l$. The additional parameters amount to only O(L·d), **requiring only a modification to the TransformerBlock.forward() method.**

From the perspective of the diffusion process: during the high-noise phase (early stage), the model relies more heavily on low-level information encoding raw perceptual features; during the low-noise phase (late stage)*, it relies more on high-level semantic representations. AttnRes achieves dynamic selection of cross-layer information through a deep attention mechanism.

From the perspective of robot control, features at different levels play distinct roles in different decision-making stages: shallow-level features correspond to low-level visual information such as edges and contact, whilst deep-level features encode task progress and semantic structure. AttnRes can dynamically fuse multi-level information during action generation, thereby better adapting to complex operational tasks.

4. Experimental

Platform： ManiSkill2/ManiSkill3 simulation。

Tasks： Choose from two tasks of increasing difficulty：

- `PickCube-v1` —— Short-duration, simple operational tasks (basic validation)
- `PegInsertion-v1` ——Precision insertion tasks requiring fine motor control (primary focus)

Baselines：

1. Original Diffusion Transformer Policy (standard additive residual)
2. Our modified AttnRes-Diffusion Transformer Policy

Ablation experiment setup：

These ablation experiments aim to distinguish the contributions of the following factors: (1) access to historical information across layers, and (2) the dynamic attention weighting mechanism. The following control experiments were designed

1. **Block AttnRes：** Applying attention after grouping layers (main experimental setup)
2. **Uniform Attention Contro：** Use the AttnRes architecture but set the attention weights to a uniform distribution (to verify the contribution of the attention mechanism itself)

For example, in layer lll:

$\alpha_{l,i} = \frac{1}{l+1}$

Implementation details: Based on the official Diffusion Transformer Policy codebase [arXiv:2410.15959], with only the following modifications: `TransformerBlock.forward()` The method used to implement AttnRes. The training hyperparameters (AdamW, cosine LR scheduling, DDPM noise scheduling) remain consistent with the original version to ensure a fair comparison.

Training budget: 300,000 training steps on a single GPU (each run is expected to take approximately 12–16 hours).

5. Evaluation criteria

Task Success Rate (%): Key metric—the proportion of rounds in which the robot completes the target task within the time limit (averaged over 100 rollouts).

Cumulative Reward per Round: Average cumulative reward per round, used to capture differences in partial completion progress.

Convergence Rate: The curve showing how success rate changes with the number of training steps, used to determine whether AttnRes leads to faster or more stable learning.

AttnRes Weight Visualisation: Analysis of how $\boldsymbol{\alpha}_l$ at each layer varies with diffusion time step k, to verify whether the model has learnt meaningful deep routing patterns.

Computational Overhead Analysis: A comparison of training time, iteration speed and GPU memory usage to evaluate the trade-off between performance gains and computational cost introduced by AttnRes.

Diffusion Time Step Analysis: An analysis of the model's behavioural changes at different diffusion time steps k to verify whether AttnRes can adaptively select information from different levels during high-noise and low-noise phases.

6. Expected Outcomes and Risk Analysis

6.1. Expected results:

`PegInsertion-v1` **The success rate of tasks has improved by 3%–8% compared to the baseline (target)**; fine motor control tasks may benefit more from reweighting at the adaptive layer. `PickCube-v1` The two tasks exhibit similar performance, confirming that AttnRes does not cause a performance drop on simple tasks.

The visualisation of AttnRes weights is expected to demonstrate that the early stages of diffusion (high noise, coarse denoising) assign higher weights to lower layers, whilst the later stages (fine denoising) rely more heavily on deeper layers —providing an interpretable narrative to support the method's effectiveness.

6.2. Risk Analysis:

(1) No performance improvement in operational tasks (Probability: Moderate)

Although AttnRes has been validated in large-scale language models, robot control involves different modalities and task structures, and its effectiveness may not be directly transferable. Even if the final performance does not show a significant improvement, we will still provide valuable qualitative analysis results by examining the distribution of attention weights and the selection patterns of layers, thereby making a contribution to the understanding of the methodology.

(2) Weighting based on layer history in softmax leads to training instability (Probability: Low)

AttnRes is inherently numerically stable; initialising $\mathbf{q}_l$ to a value close to zero ensures that the initial behaviour approximates the standard residual.

(3) Increased computational overhead (Probability: Low)

Each layer of AttnRes adds only $O(L)$ dot-product operations; the total computational overhead is expected to increase by no more than 5%.

(4) VRAM overhead from caching historical representations (Probability: Low)

The additional VRAM overhead incurred by storing historical hidden representations for each layer is limited. For a single batch, the historical representation tensor can be denoted as $\mathbf{H}_i \in \mathbb{R}^{B\times T\times d}$, where B is the batch size, T is the number of tokens, and d is the hidden dimension.

References

Chi et al., *Diffusion Policy: Visuomotor Policy Learning via Action Diffusion*, RSS 2023.

Hou et al., Diffusion Transformer Policy, arXiv:2410.15959, 2024.

Kimi Team, *Attention Residuals*, arXiv:2603.15031, 2026.

Gu et al., *ManiSkill2: A Unified Benchmark for Generalizable Manipulation Skills*, ICLR 2023.